Fetcher 페처

개요

Fetcher(페처)는 IPFS에서 DAG(Directed Acyclic Graph) 구조의 데이터를 효율적으로 가져오는 시스템입니다. Boxo의 fetcher 모듈은 로컬 및 원격 데이터 소스에서 노드를 검색하고, 선택자(selector)를 사용한 정밀한 데이터 탐색, 그리고 검증 가능한 데이터 가져오기를 지원합니다.

주요 개념

Fetcher의 역할

1. **노드 검색**: 링크를 통해 개별 노드를 가져오기
2. **그래프 탐색**: 선택자를 사용한 복잡한 DAG 탐색
3. **데이터 검증**: 가져온 데이터의 무결성 검증
4. **효율적 캐싱**: 중복 요청 방지 및 성능 최적화

핵심 구성 요소

- **Fetcher**: 데이터 가져오기 인터페이스
- **Factory**: Fetcher 세션 생성 팩토리
- **FetchResult**: 가져온 데이터와 메타데이터
- **FetchCallback**: 탐색 중 호출되는 콜백 함수

기본 사용법

Fetcher 설정

```
package main

import (
"context"
"log"

"github.com/ipfs/boxo/fetcher/impl/blockservice"
"github.com/ipfs/boxo/blockservice"
"github.com/ipfs/boxo/blockstore"
"github.com/ipfs/go-datastore"
"github.com/ipfs/go-datastore/sync"
"github.com/ipld/go-ipld-prime"
cidlink  "github.com/ipld/go-ipld-prime/linking/cid"
)

func  main() {
ctx := context.Background()
// 블록스토어와 블록서비스 설정
dstore := sync.MutexWrap(datastore.NewMapDatastore())
bstore := blockstore.NewBlockstore(dstore)
bserv := blockservice.New(bstore, nil)
```

```
// Fetcher 설정 생성

fetcherConfig := bsfetcher.NewFetcherConfig(bserv)

// Fetcher 세션 생성

fetcher := fetcherConfig.NewSession(ctx)

// 링크로 노드 가져오기

link := cidlink.Link{Cid: cid.MustParse("QmYourContent...")}

node, err := fetcher.BlockOfType(ctx, link, basicnode.Prototype.Any)

if err != nil {

log.Fatal(err)

}

log.Printf("노드를 성공적으로 가져왔습니다: %v", node)

}
```

단일 블록 가져오기

```
// 헬퍼 함수를 사용한 간단한 블록 가져오기

func fetchSingleBlock(ctx context.Context, fetcher fetcher.Fetcher, c cid.Cid)

link := cidlink.Link{Cid: c}

// 헬퍼 함수 사용

node, err := helpers.Block(ctx, fetcher, link)

if err != nil {

return nil, fmt.Errorf("블록 가져오기 실패: %w", err)

}
```

```
log.Printf("블록 %s를 성공적으로 가져왔습니다", c)

return node, nil

}


// 타입이 지정된 블록 가져오기

func fetchTypedBlock(ctx context.Context, fetcher fetcher.Fetcher, c cid.Cid, p

link := cidlink.Link{Cid: c}

node, err := fetcher.BlockOfType(ctx, link, prototype)

if err != nil {

return nil, fmt.Errorf("타입 지정 블록 가져오기 실패: %w", err)

}

return node, nil

}
```

선택자를 사용한 고급 탐색

기본 선택자 사용

```
// 모든 노드 탐색

func fetchAllNodes(ctx context.Context, fetcher fetcher.Fetcher, rootCid cid.Ci

link := cidlink.Link{Cid: rootCid}

// 모든 노드를 방문하는 콜백

callback := func(result fetcher.FetchResult) error {
```

```
log.Printf("방문한 노드: %s (경로: %s)",
result.LastBlockLink, result.Path)
// 노드 데이터 처리
if result.Node.Kind() == ipld.Kind_String {
str, _ := result.Node.AsString()
log.Printf("문자열 데이터: %s", str)
}
return nil
}
// 모든 노드 탐색
err := helpers.BlockAll(ctx, fetcher, link, callback)
if err != nil {
return fmt.Errorf("전체 탐색 실패: %w", err)
}
return nil
}
```

커스텀 선택자 구성

```
// 특정 조건에 맞는 노드만 선택
func fetchWithCustomSelector(ctx context.Context, fetcher fetcher.Fetcher, root
// 선택자 빌더 생성
ssb := builder.NewSelectorSpecBuilder(basicnode.Prototype.Any)
```

```
// 특정 필드만 선택하는 선택자

selector := ssb.ExploreFields(func(efsb builder.ExploreFieldsSpecBuilder) {

efsb.Insert("name", ssb.Matcher())

efsb.Insert("size", ssb.Matcher())

efsb.Insert("links", ssb.ExploreIndex(0, ssb.Matcher()))

}).Node()

link := cidlink.Link{Cid: rootCid}

callback := func(result fetcher.FetchResult) error {

log.Printf("선택된 노드: %s", result.Path)

return nil

}

err := helpers.BlockMatching(ctx, fetcher, link, selector, callback)

if err != nil {

return fmt.Errorf("선택자 탐색 실패: %w", err)

}

return nil

}
```

재귀적 탐색

```
// 깊이 제한이 있는 재귀적 탐색

func fetchRecursiveWithLimit(ctx context.Context, fetcher fetcher.Fetcher, root

ssb := builder.NewSelectorSpecBuilder(basicnode.Prototype.Any)
```

```
// 재귀 깊이 제한 선택자
selector := ssb.ExploreRecursive(
selector.RecursionLimitDepth(int64(maxDepth)),
ssb.ExploreAll(ssb.ExploreRecursiveEdge()),
).Node()
link := cidlink.Link{Cid: rootCid}
visitedCount := 0
callback := func(result fetcher.FetchResult) error {
visitedCount++
log.Printf("방문 #%d: %s (깊이: %d)",
visitedCount, result.LastBlockLink, len(result.Path.Segments()))
return nil
}
err := helpers.BlockMatching(ctx, fetcher, link, selector, callback)
if err != nil {
return fmt.Errorf("재귀 탐색 실패: %w", err)
}
log.Printf("총 %d개 노드를 방문했습니다", visitedCount)
return nil
}
```

검증 가능한 페칭

블록 검증

```
// 블록 무결성 검증을 포함한 페칭
func fetchWithVerification(ctx context.Context, fetcher fetcher.Fetcher, expect
link := cidlink.Link{Cid: expectedCid}
// 블록 가져오기
node, err := helpers.Block(ctx, fetcher, link)
if err != nil {
return nil, fmt.Errorf("블록 가져오기 실패: %w", err)
}
// 추가 검증 로직
if node.Kind() == ipld.Kind_Invalid {
return nil, fmt.Errorf("유효하지 않은 노드 형식")
}
log.Printf("블록 %s 검증 완료", expectedCid)
return node, nil
}

// 체크섬 검증
func fetchWithChecksumVerification(ctx context.Context, fetcher fetcher.Fetcher
link := cidlink.Link{Cid: c}
var totalSize int64
callback := func(result fetcher.FetchResult) error {
```

```
// 블록 크기 계산
if result.LastBlockPath.String() == result.Path.String() {
// 실제 구현에서는 블록 크기를 정확히 계산해야 함
totalSize += 1024 // 예시 크기
}
return nil
}
err := helpers.BlockAll(ctx, fetcher, link, callback)
if err != nil {
return fmt.Errorf("크기 검증 탐색 실패: %w", err)
}
if totalSize != expectedSize {
return fmt.Errorf("크기 불일치: 예상 %d, 실제 %d", expectedSize, totalSize)
}
log.Printf("크기 검증 완료: %d 바이트", totalSize)
return nil
}
```

신뢰할 수 있는 소스 검증

```
// 신뢰할 수 있는 피어에서만 데이터 가져오기
type trustedFetcher struct {
baseFetcher fetcher.Fetcher
```

```
trustedPeers  map[string]bool
}

func (tf *trustedFetcher) BlockOfType(ctx context.Context, link ipld.Link, proto
// 실제 구현에서는 피어 검증 로직 필요
log.Printf("신뢰할 수 있는 소스에서 블록 가져오기: %s", link)
return  tf.baseFetcher.BlockOfType(ctx, link, prototype)
}

func (tf *trustedFetcher) NodeMatching(ctx context.Context, root ipld.Node, sele
// 래핑된 콜백으로 추가 검증
wrappedCallback := func(result fetcher.FetchResult) error {
log.Printf("신뢰할 수 있는 소스에서 가져온 노드: %s", result.Path)
return  cb(result)
}
return  tf.baseFetcher.NodeMatching(ctx, root, selector, wrappedCallback)
}

// 다른 인터페이스 메서드들도 유사하게 구현...
```

성능 최적화 기법

병렬 페칭

```
// 여러 블록을 병렬로 가져오기
func fetchParallel(ctx context.Context, fetcher fetcher.Fetcher, cids []cid.Cid
type result struct {
index int
node ipld.Node
err error
}
resultCh := make(chan result, len(cids))
// 고루틴으로 병렬 처리
for i, c := range cids {
go func(index int, targetCid cid.Cid) {
link := cidlink.Link{Cid: targetCid}
node, err := helpers.Block(ctx, fetcher, link)
resultCh <- result{index: index, node: node, err: err}
}(i, c)
}
// 결과 수집
nodes := make([]ipld.Node, len(cids))
var errors []error
for i := 0; i < len(cids); i++ {
res := <-resultCh
```

```
if res.err != nil {
errors = append(errors, res.err)
continue
}
nodes[res.index] = res.node
}
if len(errors) > 0 {
return nil, fmt.Errorf("병렬 페칭 중 %d개 오류 발생", len(errors))
}
return nodes, nil
}
```

캐시 활용

```
// 캐시를 활용한 효율적인 페칭
type cachedFetcher struct {
baseFetcher fetcher.Fetcher
cache map[string]ipld.Node
cacheMutex sync.RWMutex
}

func newCachedFetcher(base fetcher.Fetcher) *cachedFetcher {
return &cachedFetcher{
```

```
baseFetcher: base,
cache: make(map[string]ipld.Node),
}
}


func (cf *cachedFetcher) BlockOfType(ctx context.Context, link ipld.Link, protot
linkStr := link.String()
// 캐시 확인
cf.cacheMutex.RLock()
if cached, exists := cf.cache[linkStr]; exists {
cf.cacheMutex.RUnlock()
log.Printf("캐시에서 블록 반환: %s", linkStr)
return cached, nil
}
cf.cacheMutex.RUnlock()
// 캐시에 없으면 실제 페칭
node, err := cf.baseFetcher.BlockOfType(ctx, link, prototype)
if err != nil {
return nil, err
}
// 캐시에 저장
cf.cacheMutex.Lock()
cf.cache[linkStr] = node
```

```
cf.cacheMutex.Unlock()

log.Printf("블록을 캐시에 저장: %s", linkStr)

return node, nil

}
```

배치 처리

```
// 배치 단위로 효율적인 처리

func fetchInBatches(ctx context.Context, fetcher fetcher.Fetcher, cids []cid.Ci

for i := 0; i < len(cids); i += batchSize {

end := i + batchSize

if end > len(cids) {

end = len(cids)

}

batch := cids[i:end]

log.Printf("배치 처리 중: %d-%d/%d", i+1, end, len(cids))

// 배치 내에서 병렬 처리

_, err := fetchParallel(ctx, fetcher, batch)

if err != nil {

log.Printf("배치 %d-%d 처리 실패: %v", i+1, end, err)

continue

}
```

```
// 배치 간 간격 (선택적)
if end < len(cids) {
time.Sleep(100 * time.Millisecond)
}
}
return nil
}
```

블록 방문자 패턴

고유 블록만 방문

```
// 중복 블록 방문 방지
func fetchUniqueBlocks(ctx context.Context, fetcher fetcher.Fetcher, rootCid ci
link := cidlink.Link{Cid: rootCid}
// 고유 블록만 방문하는 콜백
uniqueCallback := helpers.OnUniqueBlocks(func(br helpers.BlockResult) error {
cidLink := br.Link.(cidlink.Link)
log.Printf("고유 블록 방문: %s", cidLink.Cid)
// 블록별 처리 로직
return processBlock(br.Node, cidLink.Cid)
})
err := helpers.BlockAll(ctx, fetcher, link, uniqueCallback)
```

```go
if err != nil {
return fmt.Errorf("고유 블록 탐색 실패: %w", err)
}
return nil
}


func processBlock(node ipld.Node, c cid.Cid) error {
log.Printf("블록 처리 중: %s (종류: %s)", c, node.Kind())
switch node.Kind() {
case ipld.Kind_Map:
return processMapNode(node)
case ipld.Kind_List:
return processListNode(node)
case ipld.Kind_String:
str, _ := node.AsString()
log.Printf("문자열 블록: %s", str)
case ipld.Kind_Bytes:
bytes, _ := node.AsBytes()
log.Printf("바이트 블록: %d 바이트", len(bytes))
}
return nil
}
```

```go
func processMapNode(node ipld.Node) error {
iter := node.MapIterator()
for !iter.Done() {
key, value, err := iter.Next()
if err != nil {
return err
}
keyStr, _ := key.AsString()
log.Printf("맵 항목: %s", keyStr)
// 값 처리
_ = value
}
return nil
}


func processListNode(node ipld.Node) error {
iter := node.ListIterator()
index := 0
for !iter.Done() {
_, value, err := iter.Next()
if err != nil {
return err
}
```

```
log.Printf("리스트 항목 [%d]", index)

index++

// 값 처리

_ = value

}

return nil

}
```

블록 통계 수집

```
// 블록 탐색 중 통계 수집

type blockStats struct {

totalBlocks int

totalSize int64

blockTypes map[ipld.Kind]int

maxDepth int

}


func collectBlockStats(ctx context.Context, fetcher fetcher.Fetcher, rootCid ci

stats := &blockStats{

blockTypes: make(map[ipld.Kind]int),

}

link := cidlink.Link{Cid: rootCid}
```

```
callback := func(result fetcher.FetchResult) error {
stats.totalBlocks++
// 깊이 계산
depth := len(result.Path.Segments())
if depth > stats.maxDepth {
stats.maxDepth = depth
}
// 블록 타입 통계
kind := result.Node.Kind()
stats.blockTypes[kind]++
// 크기 추정 (실제 구현에서는 정확한 크기 계산 필요)
stats.totalSize += estimateNodeSize(result.Node)
return nil
}
err := helpers.BlockAll(ctx, fetcher, link, callback)
if err != nil {
return nil, fmt.Errorf("통계 수집 실패: %w", err)
}
return stats, nil
}


func estimateNodeSize(node ipld.Node) int64 {
switch node.Kind() {
```

```
case ipld.Kind_String:
str, _ := node.AsString()
return int64(len(str))
case ipld.Kind_Bytes:
bytes, _ := node.AsBytes()
return int64(len(bytes))
default:
return 256 // 기본 추정 크기
}
}

func printBlockStats(stats *blockStats) {
log.Printf("블록 통계:")
log.Printf(" 총 블록 수: %d", stats.totalBlocks)
log.Printf(" 총 크기: %d 바이트", stats.totalSize)
log.Printf(" 최대 깊이: %d", stats.maxDepth)
log.Printf(" 블록 타입별 분포:")
for kind, count := range stats.blockTypes {
log.Printf(" %s: %d개", kind, count)
}
}
```

오프라인 모드 지원

오프라인 페칭 설정

```
// 오프라인 모드에서 사용할 수 있는 Fetcher 설정
func setupOfflineFetcher(bserv blockservice.BlockService) fetcher.Fetcher {
fetcherConfig := bsfetcher.NewFetcherConfig(bserv)
// 오프라인 모드 활성화 - 찾을 수 없는 블록 건너뛰기
fetcherConfig.SkipNotFound = true
ctx := context.Background()
return fetcherConfig.NewSession(ctx)
}

// 부분적 DAG 탐색 (일부 블록이 없어도 계속 진행)
func fetchPartialDAG(ctx context.Context, fetcher fetcher.Fetcher, rootCid cid.
link := cidlink.Link{Cid: rootCid}
foundBlocks := 0
skippedBlocks := 0
callback := func(result fetcher.FetchResult) error {
foundBlocks++
log.Printf("발견된 블록: %s", result.LastBlockLink)
return nil
}
```

```
err := helpers.BlockAll(ctx, fetcher, link, callback)

if err != nil {

// traversal.SkipMe 오류는 정상적인 오프라인 동작

log.Printf("부분적 탐색 완료: %d개 블록 발견, %d개 블록 건너뜀",

foundBlocks, skippedBlocks)

return nil

}

log.Printf("완전한 탐색 완료: %d개 블록 발견", foundBlocks)

return nil

}
```

모범 사례

1. 리소스 관리

```
// 컨텍스트 타임아웃과 취소 처리

func fetchWithTimeout(rootCid cid.Cid, timeout time.Duration) error {

ctx, cancel := context.WithTimeout(context.Background(), timeout)

defer cancel()

// Fetcher 설정

dstore := sync.MutexWrap(datastore.NewMapDatastore())

bstore := blockstore.NewBlockstore(dstore)

bserv := blockservice.New(bstore, nil)
```

```
fetcherConfig := bsfetcher.NewFetcherConfig(bserv)

fetcher := fetcherConfig.NewSession(ctx)

link := cidlink.Link{Cid: rootCid}

callback := func(result fetcher.FetchResult) error {

select {

case <-ctx.Done():

return ctx.Err()

default:

log.Printf("처리 중: %s", result.Path)

return nil

}

}

err := helpers.BlockAll(ctx, fetcher, link, callback)

if err != nil {

if ctx.Err() == context.DeadlineExceeded {

log.Println("페칭 타임아웃")

return fmt.Errorf("페칭 타임아웃: %w", err)

}

return fmt.Errorf("페칭 실패: %w", err)

}

return nil

}
```

2. 오류 처리

```
// 견고한 오류 처리
func robustFetch(ctx context.Context, fetcher fetcher.Fetcher, rootCid cid.Cid)
link := cidlink.Link{Cid: rootCid}
retryCount := 0
maxRetries := 3
callback := func(result fetcher.FetchResult) error {
// 각 노드 처리 시 오류 처리
defer func() {
if r := recover(); r != nil {
log.Printf("노드 처리 중 패닉 복구: %v", r)
}
}()
err := processNodeSafely(result.Node)
if err != nil {
log.Printf("노드 처리 오류 (계속 진행): %v", err)
// 개별 노드 오류는 전체 탐색을 중단하지 않음
}
return nil
}
for retryCount < maxRetries {
err := helpers.BlockAll(ctx, fetcher, link, callback)
```

```
if err == nil {
return nil
}
retryCount++
log.Printf("페칭 재시도 %d/%d: %v", retryCount, maxRetries, err)
// 재시도 전 대기
select {
case <-time.After(time.Duration(retryCount) * time.Second):
case <-ctx.Done():
return ctx.Err()
}
}
return fmt.Errorf("최대 재시도 횟수 초과")
}


func processNodeSafely(node ipld.Node) error {
// 안전한 노드 처리 로직
if node == nil {
return fmt.Errorf("nil 노드")
}
// 노드 타입별 안전한 처리
switch node.Kind() {
case ipld.Kind_Invalid:
```

```
return fmt.Errorf("유효하지 않은 노드")
default:
log.Printf("노드 처리 완료: %s", node.Kind())
}
return nil
}
```

3. 메모리 효율성

```
// 메모리 효율적인 대용량 DAG 처리
func fetchLargeDAGEfficiently(ctx context.Context, fetcher fetcher.Fetcher, roo
link := cidlink.Link{Cid: rootCid}
processedCount := 0
batchSize := 1000
callback := func(result fetcher.FetchResult) error {
processedCount++
// 배치 단위로 메모리 정리
if processedCount%batchSize == 0 {
runtime.GC()
log.Printf("처리된 노드: %d개 (메모리 정리 실행)", processedCount)
}
// 메모리 사용량 모니터링
var m runtime.MemStats
```

```
runtime.ReadMemStats(&m)

if  m.Alloc > 500*1024*1024 { // 500MB 초과시

log.Printf("메모리 사용량 높음: %d MB", m.Alloc/1024/1024)

runtime.GC()

}

return  nil

}

err := helpers.BlockAll(ctx, fetcher, link, callback)

if  err != nil {

return  fmt.Errorf("대용량 DAG 처리 실패: %w", err)

}

log.Printf("대용량 DAG 처리 완료: 총 %d개 노드", processedCount)

return  nil

}
```

문제 해결

일반적인 문제와 해결책

1. **블록을 찾을 수 없음**

```
// SkipNotFound 옵션 사용

fetcherConfig.SkipNotFound = true
```

2. **메모리 부족**

```
// 배치 처리와 정기적 GC
if processedCount%1000 == 0 {
runtime.GC()
}
```

3. **네트워크 타임아웃**

```
// 적절한 타임아웃 설정
ctx, cancel := context.WithTimeout(ctx, 30*time.Second)
defer cancel()
```

Fetcher는 IPFS에서 효율적이고 안전한 데이터 검색을 위한 핵심 도구입니다. 적절한 선택자 사용과 성능 최적화를 통해 복잡한 DAG 구조도 효과적으로 탐색할 수 있습니다.